CONTENTS

1. At a glance
2. Plain-language glossary
3. The aircraft and the problem
4. The automatic flight, step by step
5. How a controller "thinks"
6. Software in the loop (SITL)
7. Hardware in the loop (HITL)
8. How we measured the trembling
9. What we found, in order
10. Every parameter
11. The code, explained
12. What is still open
13. Before the real flight
14. Where the files are

ATLAS V3 · PIXHAWK 6X PRO · PX4 V1.17 · 6 TO 9 OCTOBER 2026

ATLAS V3 Nose Lift

What we changed in the ATLAS V3 flight software and why, how we tested it first in a computer simulation (SITL) and then on the real flight computer (HITL), and what every piece of code does. Written for a reader with no technical background: every term is explained the first time it appears.

01

At a glance

The ATLAS V3 sits on the ground with its nose down, but flies with its nose 8.5 degrees up. To take off it must first raise its nose while standing on its legs, then hand control over to the autopilot, climb, hover, come down, and lower its nose again. That sequence is run by a program of ours inside the flight computer, called **nose_lift**.

The goal of these days was a flight with **no trembling** and **no sudden jolts**. We tested it in hundreds of simulated flights, and at the end on the real flight computer (the Pixhawk) connected to a laptop.

Trembling (lower is better)

0.58 → 0.19

degrees per second, average of the flights on the Pixhawk

Worst sudden movement

12.4 → 4.6

degrees per second, worst flight

Nose touches its front leg at

8.4 → 0.7

degrees per second

Sideways kick at lift-off

3–7 → ≤1.4

degrees per second

Two kinds of change got us there:

- **Code changes (firmware):** four fixes to how PX4 and nose_lift work. Two of them would also have happened on the real aircraft: the throttle reached the motors in "steps" and caused trembling, and PX4 "pushed" against the legs and then kicked at lift-off.
- **Setting changes (parameters):** how sharply the autopilot reacts, how slowly it touches the ground, how firmly the nose is held. Each one is explained in section 10.

Important: right now the Pixhawk is set up for testing on the computer (HITL) and **must not go into the aircraft like this**. The restore in section 13 has to be done before a real flight.

02

Plain-language glossary

The terms you will meet, in the order you need them.

Pixhawk	The aircraft's "brain": a small computer that reads the sensors and decides, hundreds of times a second, how hard to spin each motor.
PX4	The autopilot program that runs inside the Pixhawk. It is open source and used by thousands of drones. We added pieces of our own to it.
Firmware	The program stored permanently inside the Pixhawk. To change it we "build" it on the computer and "flash" it (copy it) over USB.
Parameter	A setting of the firmware, for example "how fast to climb". It can be changed without rebuilding the firmware, the way you change the volume on a radio without opening it. Parameters have names like `MC_PITCH_P`.
nose_lift	Our own program inside PX4. It raises the nose, runs the automatic takeoff, hover and landing, and lowers the nose again.
Pitch, roll, yaw	The three ways an aircraft tilts. **Pitch**: nose up or down. **Roll**: leaning left or right. **Yaw**: turning around the vertical axis, like a spinning top.
Degrees per second (°/s)	How fast the aircraft rotates. 1 °/s is very slow, almost invisible. 10 °/s is clearly seen as a "jolt". The nose lowering is designed to run at 3 °/s.
Hz (hertz)	How many times per second something repeats. A 3 Hz tremble goes back and forth three times a second.
Trembling	A fast back-and-forth (2 to 10 times a second) that the eye sees as "shivering". We measure it in °/s.
Jolt	One sudden movement, for example the moment the legs leave the ground. We measure it as the highest rotation speed in each phase of the flight.
Vibration	A very fast shaking (hundreds of times a second) from motors that are not perfectly balanced. The eye cannot see it, the sensors "feel" it. It is a mechanical effect, not a matter of settings.
Sensors	**Gyroscope**: measures how fast the aircraft rotates. **Accelerometer**: measures acceleration and gravity, so it "knows" where down is. **Magnetometer**: a compass. **Barometer**: height from air pressure.

H-FLOW	A small camera that looks at the ground and notices if the aircraft is sliding (optical flow). It also has a range finder that measures the distance to the ground to within a centimetre.
Estimator (EKF)	PX4's "inner sense": it combines all the sensors into its best guess of where the aircraft is, how it is tilted and how fast it moves. It can make small mistakes, for example believe the aircraft leans 0.3° while it is level.
Setpoint	What a controller wants to happen, for example "I want 8.5° of pitch". The controller compares the setpoint with reality and corrects the difference (the "error").
Throttle	The "gas": how hard all the motors spin together. The pilot gives it with the left stick of the radio. In the automatic flight nose_lift gives it.
Stabilized / Position	Two PX4 flight modes. **Stabilized**: PX4 only keeps the attitude (no tilting), the pilot gives the throttle. **Position**: PX4 also holds its position and height by itself, and the sticks become speed commands ("climb at 0.3 m/s").
Land detector	The part of PX4 that decides whether the aircraft is on the ground or flying. While it thinks "on the ground", PX4 keeps the memory of its controllers frozen (see "I" in section 5).
Allocator	The part of PX4 that splits a controller's wish (for example "nose up") into a command for each of the 9 motors.
uORB message	The parts of PX4 talk to each other with small "letters" called uORB messages. nose_lift sends one of its own, `nose_lift_output`.
MAVLink	The "language" the Pixhawk uses to talk to computers, radios and simulators, over USB or wirelessly.
SITL	Software in the loop: the flight software runs as a program *inside the computer*, and a physics simulator feeds it fake sensors. Fast, safe, thousands of flights.
HITL	Hardware in the loop: the *real* Pixhawk runs the real firmware, but its sensors are again supplied by the simulator over USB. The motors get no signal. It tests the real processor, the real delays and the board's real settings.
Seed	A number that changes the "noise" of the fake sensors. The same flight with a different seed shows whether a result is steady or just luck.

Newton (N)	The unit of force. The three nose fans together give about 36 N of thrust in normal condition. We also tested 32, 30 and 28 N to see what happens if they are weaker (for example with a tired battery).

03

The aircraft and the problem

How it is built

The ATLAS V3 has 9 fans. Six are at the back, in the wings, and do most of the work. Three are at the front, in the nose, and their main job is to raise and hold the nose. It has three legs: two at the back and one at the front, under the nose.

On the ground it sits nose-down, about 10 to 16 degrees down. In the air it must be 8.5 degrees nose *up*, because at that angle the fans push the air straight down.

Why PX4 alone is not enough

PX4 knows only one thing: "the correct attitude is the hover attitude". If you arm it while the aircraft sits nose-down, it sees a huge 20-degree "error" and immediately gives every motor full power to fix it. That is dangerous.

So we wrote **nose_lift**, which takes charge of the aircraft while it is on the ground:

1. After arming, it keeps every motor stopped.
2. When the pilot flips the "nose lift" switch, it starts only the 3 nose fans and raises the nose slowly, at 3 °/s, to 8.5°.
3. It holds the nose there, with the aircraft standing on its two rear legs.
4. It gradually hands control to PX4 (the "handover") and PX4 takes off.
5. After landing it takes control back, lowers the nose slowly onto the front leg and disarms.

> *Like lifting a wheelbarrow by its handles: first you lift, then you hold it balanced, and only then do you start pushing.*

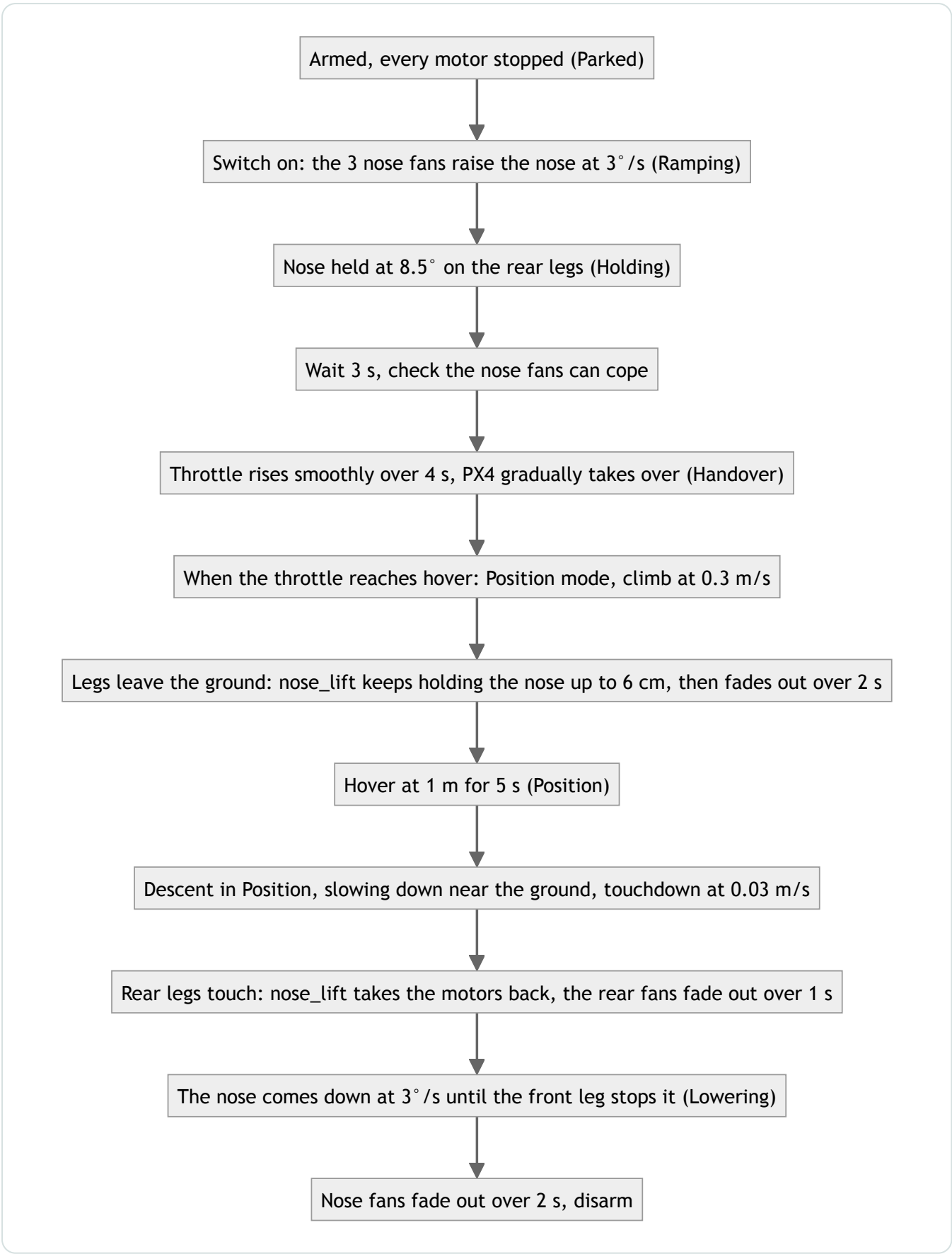
What had happened before

The first real flights (6 and 7 October) had three problems. The aircraft climbed much higher than it should, because it was in the wrong flight mode (Position instead of Stabilized). The nose stayed stuck and then "ran away" upwards. And the nose dropped sharply at landing. From then on we worked first in simulation (7 and 8 October) and then on the Pixhawk (9 October).

04

The automatic flight, step by step

This is the flight we tested every time (scenario `fw_auto_hop_winner`): a small automatic "hop" to 1 metre, with a 5-second hover.



Every phase has "safety brakes". The pilot turning the switch off lands at once, the kill switch stops everything, and nose_lift cancels if the nose cannot hold or drops suddenly.

05

How a controller "thinks"

Almost every parameter we changed tunes a **controller**. A controller does the same thing over and over: it looks at the difference between what it wants and what is happening (the error), and decides how much correction to give. The correction is made of a few "ingredients":

INGREDIENT	WHAT IT DOES	EVERYDAY EXAMPLE	IF IT IS TOO LARGE
P (proportional)	The bigger the error, the stronger the correction.	You turn the steering wheel more the further you have drifted out of your lane.	It over-corrects, swings back and forth, trembles.
I (integral, "memory")	Remembers a small error that persists and slowly adds more and more correction.	If the car keeps pulling right in a side wind, you learn to hold the wheel a little to the left.	It is slow to "forget": once the problem is gone it keeps pushing (wind-up).
D (derivative, "brake")	Reacts to how fast things change and brakes fast movements.	Just before stopping at a traffic light you ease off the brake so the car does not "dive".	It amplifies sensor noise.
Feed-forward	Gives a correction *before* an error appears, because it "knows" it will be needed.	You press the gas a little before the hill, not after the car has slowed down.	If the model is wrong, it pushes in the wrong direction.

In PX4 the controllers are **nested like Russian dolls**. The outer one (position) says "I want you to lean 1° forward". The middle one (attitude, `MC_PITCH_P`) turns that into "rotate at 4 °/s". The inner one (rate, `MC_PITCHRATE_P/I/D`) decides how much thrust difference to give the motors. If an inner controller is too "nervous", all the outer ones tremble with it.

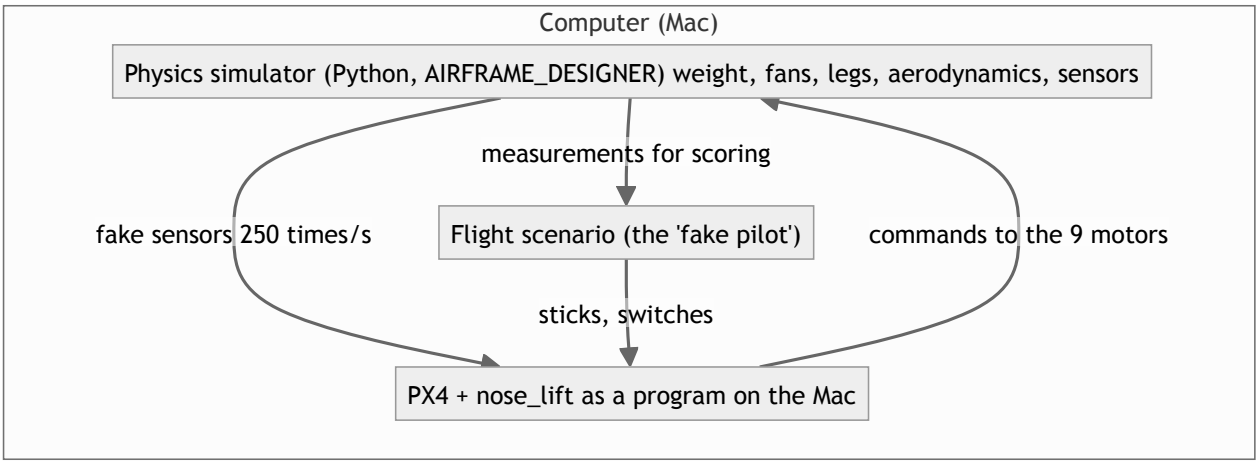
> **Why delays matter:** every controller reacts to something that has already happened. If the information arrives a little late (for example over a USB cable), a nervous controller corrects "blindly" and starts to oscillate. That is why we had to soften the roll control in HITL.

06

Software in the loop (SITL)

The idea

Instead of flying the real aircraft and taking risks, we build a "fake aircraft" inside the computer and let the real PX4 program (with nose_lift inside) fly it.



The pieces

1. **The aircraft model** (`airframes/atlas_v3_30_*.json`): mass, centre of gravity, the position and thrust of every fan, spring legs, wings. It was built from the CAD file `airframes/cad/SMALL_SCALE_V3_30.step` .

2. **The physics simulator** (`airframe_designer/sim/simulator.py` and `dynamics`): at every step it takes the motor commands, works out forces and torques (thrust, weight, air, legs on the ground) and moves the aircraft. It runs 250 sensor steps per second, with 2 physics sub-steps each.

3. **The fake sensors** (`airframe_designer/sensors/`): gyroscope, accelerometer, magnetometer, barometer, and the H-FLOW mounted at the same tilt as on the real aircraft. They have realistic noise and vibration from the imperfect balance of the fans.

4 **PX4**: the same firmware that goes to the Pixhawk, built for the Mac (`~/PX4-nl/build/px4_sitl_default`). It talks to the simulator in MAVLink: it receives `HIL_SENSOR` (sensors) and answers with `HIL_ACTUATOR_CONTROLS` (motor commands).

5 **Lockstep**: simulator and PX4 walk together, like a three-legged race. The simulator takes one step, waits for PX4's answer, then takes the next. So simulated time does not depend on how busy the computer is, and the flight runs much faster than real time (5 to 10 times).

6 **The scenario** (`scenarios/fw_auto_hop_winner.json` , `airframe_designer/sim/scenario.py`): the "fake pilot". It switches the radio on, arms, flips the nose-lift switch, waits for each phase and checks that everything happened correctly (otherwise the flight is scored FAIL or CRASH).

7 **The recorder** (`airframe_designer/sim/metrics.py`): 50 times a second it records position, speed, angles, rotation speeds, motor commands, nose_lift's state, legs on the ground, PX4's own estimate and what the controller asks for.

Many flights at once: the SITL "packs"

One flight does not tell much, because there is randomness. So we wrote `scripts/sitl_pack.py` , which runs many flights together:

- It takes a list of "candidates" (for example "baseline" and "baseline + MC_PITCH_P 3") and flies each one with several nose-fan thrusts (36, 32, 30, 28 N) and several seeds.
- It runs up to 9 PX4s at the same time, each in its own "slot" (instance) on the computer: about 20 to 30 flights a minute.
- It scores every flight (`score_run`): trembling, jolts per phase, height, sliding, touchdown speed, whether the legs touched first, how smoothly the nose came down.
- It writes everything to `results/sitl_pack/<name>/summary.json` .

In total we ran roughly 1,000 simulated flights on 8 and 9 October.

What SITL cannot show

- The real Pixhawk processor and its real delays.
- The settings already on the board from calibration and the radio.
- The real legs rocking on the ground with the fans running, and battery voltage sag.

HITL covers the first two. Only a real flight covers the third.

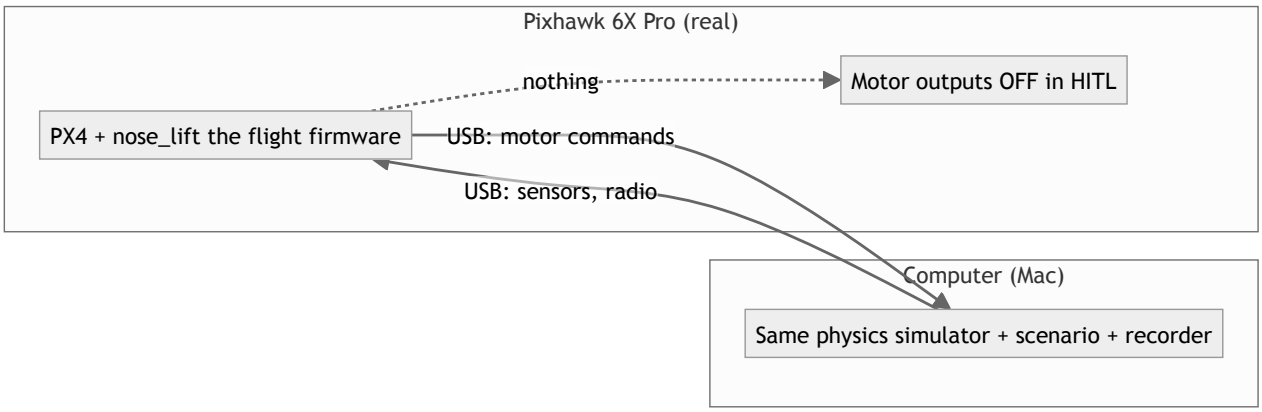
> **A simulator bug we found (8 Oct):** the simulator had an old nose-lowering routine of its own. Above 1 m of height it took over every landing instead of the firmware. It has been switched off in every test since (`design.nose_lower.enabled = false`), so that what we measure is the firmware's work only.

07

Hardware in the loop (HITL)

The idea

The same simulator, but instead of PX4 on the Mac it talks to the **real Pixhawk** over the USB cable. The Pixhawk "believes" it is flying: it gets sensors from the simulator and sends motor commands back. The real motors get nothing.



How it is made safe

With the parameter `SYS_HITL = 1`, PX4 does **not** start the programs that drive the real motors (`pwm_out`, `dshot`) when it boots. It starts only a "virtual" output program (`pwm_out_sim`) that sends the commands to the computer. We checked this in PX4's start-up script (`rcS`) before turning HITL on. The Pixhawk was also outside the aircraft.

Differences from SITL

- **Real time:** the Pixhawk has its own clock and does not wait for the simulator. A flight lasts as long as in reality, about 1.5 minutes including the reboot.
- **One flight at a time:** we have one Pixhawk.
- **The computer must be idle:** if builds or SITL packs run at the same time, the simulator falls behind and the results are spoiled. We learned this the hard way in round h1.
- **The Pixhawk reboots before every flight**, so its estimator starts clean with the aircraft resting on its legs.

The steps we took

1. **Backup:** we read and saved all 1,237 parameters of the Pixhawk (`results/board_params/params_20261008_before_hitl.json`).

2. **Firmware:** we flashed the final SITL firmware over USB (26 seconds, with verification). After every code fix we rebuilt and re-flashed: four times in total.

3. **HITL on** (`SYS_HITL 1`, reboot) and "push" of the model's parameters to the Pixhawk: motor geometry, nose_lift settings, H-FLOW settings. That was 242 parameters.

4. **We cleared three obstacles** that do not exist in SITL and that blocked arming (details below).

5. **Headless runner** (`scripts/hitl_pack.py`): it starts each flight through the app (no screen), waits for it to finish, reads the recording and scores it with the same metrics as SITL, plus the new "trembling" metric.

6. **Test rounds h0 to h6:** each round compared 2 to 5 variants, with 2 to 4 flights each. Candidate changes were always checked quickly in SITL first.

The HITL obstacles and how they were solved

WHAT HAPPENED	WHY (SIMPLY)	FIX, FOR HITL ONLY
PX4 refused to arm: "RC3_TRIM < MIN".	The scenario said "throttle middle = 1000", but the board's real calibration says "minimum = 1001". PX4 sees something impossible and refuses.	The scenario also sets `RC3_MIN 1000`. In SITL nothing changes, because it is already 1000 there.
Constant "Arming denied".	The board has an arm switch on channel 8 of the real radio. The scenario does not send channel 8, so the "switch" was always off.	`RC_MAP_ARM_SW 0` (no arm switch).
PX4 did not see the simulator's sticks at all ("No manual control input").	PX4 reads the radio only from the *first* source. The first source was the ExpressLRS receiver (CRSF) on port TELEM2, which with no receiver attached reports "signal lost". The simulator was the second source and was ignored.	`RC_CRSF_PRT_CFG 0` (CRSF off), so the simulator becomes the first source. We first suspected the IO co-processor, but it was not the cause.

WHAT HAPPENED	WHY (SIMPLY)	FIX, FOR HITL ONLY
Every flight started with a different error in heading (1 to 4°) and in level (0.2 to 0.85°).	PX4 has an "automatic calibration" of magnetometer and gyroscope that learns corrections in flight and saves them. In HITL, without lockstep, it learned wrong corrections for the fake sensors.	`MBE_ENABLE 0`, `SENS_IMU_AUTOCAL 0`, `SENS_BAR_AUTOCAL 0`, and zeroing the corrections (`CAL_MAG3_*`, `CAL_GYRO3_*`, `CAL_BARO2_OFF`) that belong only to the simulator's sensors.
The Pixhawk "froze" and you had to unplug and replug the USB.	I sent a very long line of text through its shell to write a file on the SD card.	In the end no file was needed. Recorded in the brain as a lesson: only short commands in the shell.

> All these changes are **for HITL only** and must be undone before a real flight. There is a ready restore file (section 13).

08

How we measured the trembling and the jolts

To improve something you have to measure it the same way every time. Every flight produces these numbers:

METRIC	WHAT IT MEASURES, SIMPLY	HOW IT IS COMPUTED
Trembling (trem_rms)	How much the aircraft "shivers" over the whole flight.	From the pitch and roll rotation speeds we keep only the fast back-and-forth (2 to 10 times a second) and take the average of its size.
Worst second	The shakiest second of the flight, and when it happened.	The same, in a sliding 1-second window.
Jolts per phase	The sharpest movement in each phase: handover, lift-off, climb, hover, descent, touchdown, nose lowering.	The highest pitch and roll rotation speed inside each phase.
Vibration	How much the sensors "buzz" from the motors.	PX4's own vibration metrics for accelerometer and gyroscope, during the hover.
Drift	How far the aircraft moved from where it took off.	Horizontal distance, in the hover and over the whole flight.
Touchdown	How fast it touched down, and whether the legs touched first.	Vertical speed at the moment of contact.
Nose lowering	How smoothly the nose came down and how fast the front leg touched.	Highest speed, deviation from 3 °/s, speed at the moment of contact.

The measurements have "noise" of ±1 to 2 °/s from flight to flight. So we always judged on 2 to 4 flights per variant and never on one.

09

What we found, in order

The work went like an investigation: measure, find the biggest problem, understand where it comes from, fix it, measure again.

Before HITL (7 and 8 October, SITL)

- **Climbing at a controlled speed instead of with raw throttle.** In Stabilized the aircraft slid metres forward during the climb, and the sharp braking at the hover was the worst jolt (20 to 38 °/s). Now, as soon as the throttle reaches hover level, nose_lift switches PX4 to Position and asks for "climb at 0.3 m/s".
- **Height from the range finder.** PX4's estimator "raised" the height slowly by 30 to 40 cm per flight because of the vibration. Height is now measured by the H-FLOW range finder.
- **Gentle landing.** Touchdown at 5 cm/s, nose_lift takes over as soon as the legs touch, the rear fans fade out over 1 s, and the nose comes down with no "step".
- **Softer pitch** (`MC_PITCH_P` 6 → 4) against the first climb tremble.

On HITL (9 October), round by round

ROUND	WHAT WE TESTED	WHAT WE FOUND	WHAT WE DID
h0	The final SITL set as it was.	After the HITL obstacles, 2/2 flights passed. But in one of them the nose **fell onto its leg at 12.4 °/s**. Trembling was 0.53 to 0.62.	nose_lift lowered the nose to the angle PX4 "believed" it had before the lift. A few seconds after a reboot that estimate is 1 to 2° off, so the fans switched off with the nose 1.4° above the leg. **Firmware fix: NL_LOW_VMIN.**
h1	5 variants of the gains.	The trembling was in the handover and the climb, not in the hover. The data showed **the throttle rising like a staircase**: one step every 0.3 s, and each step jolted the nose. The same was present in SITL.	PX4 recomputed the throttle only when a radio channel changed, or every 300 ms. With still sticks, nose_lift's smooth ramp became a staircase. **Firmware fix: patch 0003.** Trembling halved. We stopped the round, since it was testing the old firmware.
h2	New firmware + 3 combinations for the lift-off.	Trembling fell to 0.31 to 0.41, but was still twice the SITL value. A per-frequency analysis showed the difference was almost all in **roll, at 2 to 4 Hz**.	The small extra delay of the USB link brought the roll controller close to its stability limit.
h3	Softer roll.	With `MC_ROLL_P 4` and `MC_ROLLRATE_P 0.45` trembling fell to 0.18 in both flights. A **sideways kick at lift-off** remained (3 to 7 °/s), different in every flight.	On the legs PX4 thought the aircraft leaned 0.2 to 0.85°. We found this came from wrong "automatic calibrations" of the fake sensors.

ROUND	WHAT WE TESTED	WHAT WE FOUND	WHAT WE DID
h4	Clean sensors + 3 variants.	Corrections stayed at zero and trembling fell to 0.15 to 0.25. The sideways kick got smaller but **tracked the level error**: 0.4° of error gave 6 °/s.	The mechanism is general: while standing on its legs, PX4 tries to correct an angle it cannot change, "loads" the legs, and that load is released when they leave the ground. It would happen on the real aircraft too. **Firmware fix: patch 0004.**
h5	Full set + touchdown at 0.03 m/s.	In 4 of 6 flights every jolt was ≤ 3.4 °/s.	We checked patch 0004 in SITL with an artificial level error: from 5 °/s down to ≤ 1.2 °/s.
h6	Final firmware and set (final3), 4 flights.	4/4 passed. Trembling 0.12 to 0.27, worst jolt 3.1 to 4.6 °/s. With a 0.40° level error the sideways kick was only 1.3 °/s.	The 4.6 was a single corrupted H-FLOW sample that came over USB; it exists only in HITL.

What we tried and rejected

What did not work matters as much as what we kept, so nobody tries it again:

- **Feed-forward from the rear fans to the nose** (`NL_HO_FF` 0.1 to 0.3): crashes at 30 N.
- **A stiffer nose hold during the handover** (`NL_HO_KANG` 3 and 5, and `NL_K_ANG` 2 together with patch 0004): oscillations and crashes.
- **Less D on pitch** (0.008): more trembling.
- **Even softer roll** (`MC_ROLL_P` 2.5): more roll wobble in the hover.
- **Taking off directly in Position from the legs** (`NL_AUTO_PTKO`): PX4 gives full thrust in one step.
- **Gyroscope filter**: does not change the vibration, because the vibration is mechanical.

10

Every parameter, one by one

For each parameter: what it is, the value the board had before (`old`), the final value (`new`), and why, starting from the physics of the problem. "Did not exist" means the parameter was created by us in the firmware. Parameters starting with `NL_` belong to nose_lift. `MC_` belong to PX4's attitude controller, `MPC_` to its position controller, and `COM_` to PX4's "commander".

10.1 Raising and holding the nose on the legs

The physics: to raise the nose, the nose fans must produce more torque around the rear legs than the weight does. nose_lift knows the geometry and computes a feed-forward ("how much thrust is needed here to balance"). On top of that, a rotation-speed controller corrects so the nose rises at 3 °/s.

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
`NL_KQI`	The "memory" (I) of the nose controller.	`0.012`	`0.024`	In the handover, as the rear fans spin up, the nose "sagged" slowly (8.6° → 7.7°). A stronger memory reacts faster to that slow sag. The lift-off jolt fell from 5.9 to about 4 to 5 °/s.
`NL_I_MAX`	The ceiling of that memory while the nose rises.	`did not exist`	`40`	On 7 Oct the nose stuck for 4 to 8 s, the memory "filled up", and when the nose broke free it pushed it to 24 to 27°. The ceiling, plus "freezing" the memory while the motor is at full, stops that build-up. 10 proved too small (the nose would not rise); 40 is the old value that worked.
`NL_Q_FC`	How much the rotation-speed measurement is "smoothed" before the controller uses it.	`did not exist`	`50 Hz`	We tried heavy smoothing (2 Hz) to ignore the legs rocking. It made the hold slow and loose. 50 Hz smooths practically nothing, as before.
`NL_STUCK_S`	After how many seconds of a "stuck" nose (motors at full, no movement) the raise is cancelled.	`did not exist`	`10 s`	A safety net for the 7 Oct case. You found the first proposal of 2 s too short for the nose to break free, so it is 10.
`NL_MAX_CMD`	The highest command to the nose fans.	`1.0`	`1.0`	0.8 and 0.64 were tried on 7 Oct. With them nose_lift calculated that the nose could not even be lifted, and cancelled. Back to 1.0.
`NL_WEIGHT`	The weight nose_lift "believes" when it computes the torque.	`141.24`	`141.24`	105 was tried on 7 Oct and the nose would not rise. Back to the original.
`NL_K_ANG`	How much rotation speed it asks for per degree of angle error.	`1.0`	`1.0`	2 reduced trembling on its own, but together with patch 0004 it caused crashes at 30 N. Kept at 1.

10.2 Automatic takeoff and climb

The physics: the aircraft must go from "standing on its legs, nose held by its fans" to "flying, PX4 controls every motor". If the thrust rises suddenly, the aircraft jumps. If it rises too slowly, the legs unload without flying and the nose drops (this happened on 7 Oct).

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
`NL_AUTO_RAMP`	Seconds for the throttle to rise from 0 to hover.	`0.5 s`	`4 s`	With the new climb in Position (PCLB 2), PX4 takes over at the end of the ramp, still on the legs. A slow ramp lets the nose and PX4 adapt smoothly. In SITL it was the gentlest option.
`NL_AUTO_SLEW`	How fast the automatic throttle may change.	`1.5 /s`	`0.3 /s`	Slower changes mean fewer "steps" in the thrust. This was the original value, raised to 1.5 on 7 Oct, and now restored.
`NL_AUTO_VUP`	Climb speed.	`0.5 m/s`	`0.3 m/s`	A slower climb means less trembling and less braking at the hover.
`NL_AUTO_PCLB`	When it switches to Position for the climb.	`did not exist`	`2`	2 means "as soon as the ramp reaches hover, still on the legs". The climb is then a speed command, so the aircraft does not slide and no sharp braking is needed later.
`NL_AUTO_PACC`	How fast the requested climb speed may change.	`did not exist`	`0.3`	The climb starts and ends smoothly, with no "kick".
`NL_AUTO_PTKO`	Take off directly in Position from the legs.	`did not exist`	`0 (off)`	rejected PX4 jumps the thrust from 0 to hover in one step.
`NL_AUTO_RNG`	Where the height is measured from.	`did not exist`	`1 (range finder)`	The estimator's height "rose" 30 to 40 cm per flight from vibration. The range finder measures the distance to the ground directly.
`NL_AUTO_ALT`	Hover height.	`1.0 m`	`1.0 m`	From 1.5 to 1.0 on 7 Oct, because SITL showed the fast ramp overshot the target by 0.5 m.
`NL_AUTO_HOV`	Hover duration.	`5 s`	`5 s`	Unchanged: a short test flight.
`NL_HO_LIFT` / `NL_HO_LIFT_H`	nose_lift keeps holding the nose until the aircraft has really left its legs by this height.	`did not exist`	`1 / 0.06 m`	On the legs PX4 let the nose sag 2° and then "snapped" it back at lift-off. At 6 cm we are sure it is flying. 3 cm was "fooled" by the leg springs.
`NL_HO_FF`	Feed-forward: as the rear fans rise, give the nose more in advance.	`did not exist`	`0 (off)`	rejected Any value above 0.05 caused crashes at 30 N.

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
NL_HO_KANG	A stiffer angle hold, for the handover only.	did not exist	0 (off)	rejected 3 and 5 caused oscillations and crashes. 2 gave a tiny gain and doubled the trembling. It stays in the firmware for future use.
NL_LEGS_ATT	On which axes PX4 does *not* fight against the legs (1 = roll, 2 = pitch, 3 = both).	did not exist	3	On the legs the aircraft cannot change its angle, so every PX4 "correction" only loads the legs and comes out as a jolt when they leave the ground. With a 0.4° level error, roll at lift-off fell from 5 to ≤ 1.2 °/s, and pitch from 3.8 to 1.6 to 2.9 °/s.
NL_LEGS_FADE	In how many seconds after lift-off PX4 smoothly returns to its normal target.	did not exist	0.5 s	1 s left the aircraft slightly tilted for too long; it began to slide and PX4 braked sharply later. 0.5 s gave the smallest worst jolt (≤ 4.0 °/s).

10.3 Safety brakes of the automatic flight

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
NL_AUTO_HCMD	If the nose fans need more than 83% command to hold the nose, the takeoff is cancelled.	did not exist	0.83	If the nose barely holds on the ground (for example a tired battery), it will drop in the handover. SITL showed flights fail above this value. The first value (0.71) was in the wrong unit and cancelled every takeoff.
NL_AUTO_HDROP	If in the first 3 s of the climb the nose drops more than 5° below target, the throttle is cut and the nose lowered.	did not exist	5°	Matches the 7 Oct failure, where the nose dropped during the handover.
COM_DISARM_LAND	After how many seconds on the ground PX4 disarms by itself.	-1	-1 (never)	nose_lift does the disarming, and only after lowering the nose. If PX4 disarmed earlier, the nose would drop.
COM_RC_OVERRIDE	Whether moving the sticks takes control away from the automatic modes.	1	0	An accidental touch of a stick would interrupt the automatic flight. The pilot still has the switch and the kill switch.

10.4 Hover and position hold (PX4 position controller)

The physics: to move sideways the aircraft must tilt. The position controller decides how much to tilt to stay over its spot. If it does this sharply, every small slide becomes a jolt.

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
NL_AUTO_HOLD	The hover is flown by PX4 in Position.	0	1	PX4 in Position holds height and position with its own instruments (H-FLOW), much more steadily: height varies by less than 1 cm.
MPC_ACC_HOR	Largest horizontal acceleration it asks for.	3	2	Gentler position corrections, fewer jolts.
MPC_JERK_MAX	How suddenly the acceleration may change.	8	2	"Jerk" is exactly what you feel as a kick. Smaller means silkier movements.
MPC_XY_P	How hard it returns to its spot per metre of error.	0.95	1.5	With the softer setting the aircraft slid 1 to 3 m at 30 to 28 N. A firmer position hold means less sliding; the smoothness comes from the acceleration and jerk limits.
MPC_XY_VEL_P_ACC	How hard it corrects horizontal speed.	1.8	2.4	Together with the one above: a firmer position without becoming nervous.
MPC_XY_VEL_I_ACC	The "memory" of the speed controller (for example against a steady wind or tilt).	0.4	1.0	Better resistance to a steady "push", such as the small nose error in the hover.

10.5 Aircraft attitude (PX4 attitude controller)

The physics: these controllers keep the aircraft level. If they are too "nervous" compared with how slowly the fans can change thrust (fans have inertia), they over-correct and tremble. If the information arrives late, it gets worse.

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
MC_PITCH_P	How fast it wants to correct a pitch angle.	6	3	6 caused a 2 Hz wobble in the climb ("it trembles when it climbs"). 4 halved it (8 Oct), and 3 reduced the trembling further in SITL and HITL. Like turning the wheel more calmly.
MC_PITCHRATE_D	The "brake" (D) of the inner pitch controller.	0.008	0.012	More brake damps oscillations. The test with less (0.008) on HITL gave more trembling.
MC_ROLL_P	How fast it wants to correct a roll angle.	5	4	On HITL the roll oscillated at 2 to 4 Hz because of the small USB delay. Slightly softer gives it "margin".
MC_ROLLRATE_P	The P of the inner roll controller.	0.6	0.45	Together with the one above, it cut the HITL trembling from 0.34 to 0.18 °/s, and in SITL from 0.16 to 0.12. A real aircraft always has some delays, so the margin helps there too.

10.6 Descent, touchdown, nose lowering

The physics: at 8.5° the rear legs are lower, so they touch first. They sit behind the centre of gravity, so their impact turns the nose down. The slower the aircraft touches, the smaller the impact. Afterwards, the nose must come down under control, not fall.

PARAMETER	WHAT IT IS	BEFORE	AFTER	WHY
`NL_AUTO_PDSC`	The descent is flown in Position, with a speed command from nose_lift.	`did not exist`	`1`	PX4's own "Land" touched down at 0.2 m/s and then let the nose fall uncontrolled for 5 s. Now the descent slows down near the ground (starting at 25 cm, NL_AUTO_HTD) and nose_lift takes over immediately.
`NL_AUTO_VTD`	Touchdown speed.	`did not exist`	`0.03 m/s`	Going from 0.05 to 0.03 m/s cut the impact jolt from 4.7 to 3.0 °/s, for only 3 s of extra flight. The legs touched first in 6/6 flights.
`NL_AUTO_TD`	When nose_lift considers that it has touched down.	`did not exist`	`3 (its own cue)`	It does not wait for PX4's land detector, which takes several seconds. It watches the range finder (below 6 cm) and stillness, and takes over within a fraction of a second.
`NL_AUTO_HFL` / `NL_AUTO_VFL`	Flare height and speed of its own landing (a fallback).	`did not exist`	`0.7 m / 0.15 m/s`	If it has to land by itself (for example the pilot turns the switch off), it slows down below 0.7 m.
`MPC_LAND_SPEED` / `MPC_LAND_CRWL` / `MPC_LAND_ALT3`	The speeds of PX4's own Land (a fallback).	`0.3 / 0.3 / 1.0`	`0.2 / 0.15 / 0.7`	If PX4's Land takes over for any reason, it touches down more gently.
`NL_LOW_RRAMP`	Time for the rear fans to switch off after landing.	`did not exist`	`1.0 s`	They used to switch off at once and the aircraft jolted at ±14 °/s. Now they fade out smoothly.
`NL_LOW_VMIN`	The slowest the nose may come down, until the leg stops it.	`did not exist`	`1.0 °/s`	The nose came down only to the angle PX4 "believed", while the real one was 1.4° lower. The fans switched off with the nose in the air and it fell at 12 °/s. Now it keeps coming down slowly until it really touches, and only then do the fans switch off. The leg now touches at 0.6 to 0.8 °/s.

10.7 HITL-only settings undone before flight

PARAMETER	WHAT IT IS	BOARD	IN HITL	WHY
`SYS_HITL`	HITL mode.	`0`	`1`	Turns the real motor outputs off and takes sensors from the computer.

PARAMETER	WHAT IT IS	BOARD	IN HITL	WHY
`RC_MAP_ARM_SW`	Channel of the arm switch.	`8`	`0`	The simulator does not send channel 8.
`RC_CRSF_PRT_CFG`	Port of the ExpressLRS receiver.	`102 (TELEM2)`	`0`	Otherwise the receiver "hides" the simulator's radio.
`MBE_ENABLE`, `SENS_IMU_AUTOCAL`, `SENS_BAR_AUTOCAL`	Automatic calibration of magnetometer, IMU and barometer.	`1, 1, 1`	`0, 0, 0`	In HITL they learn wrong corrections for the fake sensors.
`CAL_MAG3_*`, `CAL_GYRO3_*`, `CAL_BARO2_OFF`	Corrections of the fake sensors.	`small values`	`0`	They belong only to the simulator's sensors and are not used on the real aircraft.
`EKF2_OF_DELAY`, `EKF2_RNG_DELAY`	The H-FLOW delay the estimator "knows".	`20 / 5 ms`	`0 / 0`	The fake sensor has no delay. The real one does, and needs the old values back.
`RC_MAP_KILL_SW`, `RC3_MIN`, `RC3_TRIM`	Kill-switch channel, throttle calibration.	`5, 1001, 1001`	`9, 1000, 1000`	Set by the test scenario. **On the real aircraft the kill switch is on channel 5.**
`HIL_ACT_FUNC1 to 9`	Which virtual output is which motor.	`did not exist`	`Motor 1 to 9`	Exist only in HITL. In flight the real outputs (PWM_MAIN_FUNC) are used.

10.8 Simulator settings (not the aircraft's)

- `design.nose_lower.enabled = false`: turns off the simulator's old nose-lowering routine, so we measure only the firmware.
- `design.flow_sensor.enabled = true`: the fake H-FLOW, at the real mounting tilt (20.4°).
- `rotors[6:9].max_thrust = 36 / 32 / 30 / 28`: the real thrust of the nose fans. PX4 always "believes" 36 N, which lets us see what happens if they are weaker.
- `--accel-bias 0,0.07,0` (new): an artificial accelerometer error that makes PX4 believe it leans 0.4°. With it we reproduced the HITL sideways kick in SITL.

11

The code, explained

The code lives in two worlds. The **firmware** (written in C++) runs inside the Pixhawk and flies the aircraft. The **tools** (written in Python) run on the computer: they simulate, test, measure and show the results.

11.1 nose_lift (firmware)

Files: `firmware/px4_ext/src/modules/nose_lift/NoseLift.cpp` (about 1,850 lines), `NoseLift.hpp` (the "bill of materials": what it remembers and which parameters it reads), `params.c` (its 96 parameters, each with a description).

How it runs

PX4 wakes nose_lift up **200 times a second** (`Run()`). Each time it:

1. Reads the "letters" that have arrived: attitude and rotation speed, armed or not, on the ground or not, sticks and switches, position and height, the range finder, PX4's motor commands, the barometer and PX4's status.

2. Reads the clock *after* reading those letters. This small detail fixes a real bug: if the clock was read first, a fresher message looked as if it came "from the future", and nose_lift believed the attitude had been lost.

3. Runs its "state machine" (`step_sequence`) and, in an automatic flight, the flight sequence (`step_auto`).

4. Sends the result to the rest of PX4 (`publish`).

The states

STATE	WHAT IT DOES
`Parked`	Armed on the ground, every motor stopped.
`Rampin g`	Raises the nose at 3 °/s. Only the nose fans run.

STATE	WHAT IT DOES
`Holding`	Holds the nose at 8.5°.
`Handover`	PX4 runs every motor; nose_lift keeps a "floor" under the nose fans until PX4 reaches it by itself and the aircraft is 6 cm off its legs. Then the floor fades out over 2 s.
`Flying`	Flying: nose_lift just watches (and gives the throttle in an automatic flight).
`Lowering`	After landing, or on a cancel: lowers the nose under control onto the leg, then stops the motors and disarms.
`Aborted`	Everything stopped until disarm (kill switch, attitude lost, too much roll, nose overshot its target, and so on).

The physics inside the code

- `balance_fraction()` : computes how much thrust the nose fans need to balance the weight around the rear legs *at the current angle*. It takes the weight's torque (weight × distance from the legs) and divides it by the fans' "leverage" (`NL_A0` to `NL_A2`). This is the feed-forward.
- `thrust_to_cmd()` , `motor_thrust()` , `motor_command()` : translate "how much thrust I want" into "what command I send to the motor". Thrust is not proportional to the command; it behaves more like a square (twice the command gives roughly four times the thrust).
- `update_split()` : the 3 nose fans are slightly angled. This function shares the thrust among them so the aircraft does not turn (yaw) or lean (roll) while the nose rises.
- `lifted_off()` : checks whether the aircraft has lifted off by accident while only the nose should be rising. It requires the height, the climb speed *and* the barometer to agree, because on the real aircraft vibration falsely "raised" the height estimate.

The nose controller (inside `step_sequence`)

Every state does the same thing in three steps:

```
wanted rotation speed = K_ANG × (target - angle now)       (limited to 3 °/s)
speed error           = wanted rotation speed - rotation speed now
nose thrust           = balance feed-forward + KQ × speed error + KQI × error mem
```

The protections added on 7 Oct: the memory "freezes" while the motor is at full, it has a ceiling (`NL_I_MAX`), and it is reset to zero if the nose runs too fast. That reset looks at a smoothed rotation speed, because the legs rocking kept triggering it. There is also a cancel for a "stuck" nose after `NL_STUCK_S` .

The new lowering logic (NL_LOW_VMIN)

```
if NL_LOW_VMIN > 0:
    wanted rotation speed = at most -1 °/s          (the nose always keeps comin
    touched = (near the starting angle AND standing still while asked to come dow
              or (3° below the starting angle)
    only then → fade the fans out over 2 s
```

In plain words: we do not trust an estimated angle to know when the leg has touched. We keep coming down slowly until we "feel" that something has stopped us.

The automatic flight (`step_auto`)

- `Wait` : waits 3 s at the hold and checks the `NL_AUTO_HCMD` brake.
- `Climb` : raises the throttle over a 4 s ramp. At the end (PCLB 2) it asks PX4 for Position mode and from then on gives a "climb-speed stick" (`stick_for_climb`). That function reverses PX4's own "translation" from stick position to speed, so that it asks for exactly 0.3 m/s.
- `Hover` / `HoldWait` / `HoldPX4` : 5 s hover in Position. Height comes from `ekf_height()` : range finder minus its reading at takeoff, otherwise the estimator, otherwise the barometer.
- `Descend` : descent in Position, with a speed that shrinks as the ground gets closer, down to 0.03 m/s.
- `update_touchdown()` / `touched_down()` : its own sign of touchdown (range finder below 6 cm, and stillness).
- `lower_from_flight()` : "no step". It starts the nose fans from the command they had just before, from PX4, and lets the rear fans fade out over 1 s. It used to start from zero, and the aircraft jolted.
- Brakes: `NL_AUTO_HDROP` (nose drop in the climb), switch off in the air (smooth landing), pilot throttle above a limit (the pilot takes the throttle), kill switch.

What it sends (`publish`)

The message `nose_lift_output` (described in `firmware/px4_ext/msg/NoseLiftOutput.msg`) contains:

- which motors nose_lift "governs" and with what command, either as a replacement or as a "floor",
- the state, the angle and the target,
- the throttle of the automatic flight,
- **new:** `on_legs` (whether the aircraft stands on its legs inside the sequence), `legs_hold` and `legs_fade_s` (the NL_LEGS_ATT settings).

nose_lift **never drives the motors itself**. It sends the message, and the allocator applies it. That way PX4's kill switch and disarm, which act after it, always stop everything.

11.2 The four patches to PX4 itself

A "patch" is a small change to PX4's own code (not to our program), stored as a file in `firmware/patches/` so that it is applied again automatically on every build.

PATCH	WHERE	WHAT IT DOES, SIMPLY
`0001`	control_allocator	Just before the commands go to the motors, it looks at `nose_lift_output`. For the motors nose_lift asks for, it either puts in nose_lift's command (replacement) or keeps the larger of the two (floor). If nose_lift stops talking for more than 0.1 s, it stops the motors nose_lift owned, for safety. It also sends back to nose_lift what PX4 wanted (`nose_lift_feedback`).
`0002`	start-up script (rcS)	Starts nose_lift when the Pixhawk boots. It stays idle if `NL_EN` is 0.
`0003`	rc_update	(a) While nose_lift gives a throttle, puts it in place of the pilot's stick. (b) new 9 Oct While it gives a throttle, PX4 re-reads it on *every* radio packet (50 times a second or more), not only when a stick changes or every 300 ms. This was the cause of the climb trembling.
`0004`	mc_att_control	new 9 Oct When a new attitude target arrives and nose_lift says `on_legs`, it sets the roll or pitch target equal to the current estimate (limited to ±3°), so PX4 does not "fight" the legs. Once the aircraft leaves its legs the difference fades out smoothly (a cosine curve) over `NL_LEGS_FADE` seconds.

```
// patch 0004, simplified
if (nose_lift says on_legs):
    roll_offset  = estimated_roll  - target_roll      (at most ±3°)
    pitch_offset = estimated_pitch - target_pitch    (if NL_LEGS_ATT asks for it)
once off the legs:
    offset = starting_offset × ½(1 + cos(π × time / NL_LEGS_FADE))   → 0
new target = PX4's target + offset
```

11.3 How the firmware is built and flashed

- `scripts/build_nose_lift_firmware.sh sitl` or `board`: copies our program next to PX4 (`~/PX4-nl`), applies any missing patches and builds. On this Mac it needs `SDKROOT=…/MacOSX26.5.sdk` (the newer SDK does not work with the current linker) and the build tools from `~/.venvs/px4-build`.
- Flashing is done with `Tools/px_uploader.py --port /dev/cu.usbmodem01 …px4`. The app must be closed, because it holds the USB port.
- The Pixhawk's memory is 97.4% full. Small changes still fit, large additions do not.

11.4 The tools on the computer (Python)

FILE	WHAT IT DOES
`airframe_designer/sim/simulator.py`	The simulation loop. Each step: motor commands → physics → "hooks" (scenario, recorder) → sensors → PX4. In SITL it waits for PX4's answer (lockstep). In HITL it runs in real time.
`airframe_designer/sensors/models.py` , `flow.py` , `vibration.py`	The fake sensors: noise, errors (for example the new `accel_bias`), vibration from the fans' balance, and the H-FLOW at its real tilt.
`airframe_designer/sim/scenario.py`	The "fake pilot": runs the scenario's phases (radio, arming, switches), sends the radio channels 50 times a second, and checks whether the flight succeeded or the aircraft crashed.
`airframe_designer/sim/metrics.py`	Records the flight 50 times a second. We added: legs on the ground, nose_lift state and command, and the position and speed PX4 *estimates*.
`airframe_designer/px4/link.py`	The MAVLink "phone line" to PX4: sends sensors and radio, receives motor commands, parameters and nose_lift's status.
`airframe_designer/px4/connection.py`	Connects to SITL or HITL and builds the readiness checklist you see in the app: port, firmware, SYS_HITL, motor commands flowing, can arm, geometry.
`airframe_designer/server/app.py`	The app (a page in the browser + an API). We added a fix so the connection status does not "crash" when the board sends NaN ("empty") values.
`airframe_designer/batch/worker.py` , `cli.py`	One flight with no screen (`python -m airframe_designer run …`). New option `--accel-bias` .
`scripts/sitl_pack.py`	The SITL packs: many flights in parallel, scoring (`score_run`), summary. New: extra options per candidate (`args` , `cand_args`), and a 300 s limit per flight so one "stuck" flight does not block the pack.

FILE	WHAT IT DOES
`scripts/hitl_pack.py`	new The HITL packs: one flight at a time on the Pixhawk, through the app. Every flight sends all the baseline parameters, because the board remembers whatever was left from the previous flight. It also contains the trembling metric (`tremble`).
`scripts/sitl_trem_table.py` , `sitl_pack_report.py` , `sitl_pack_table.py`	Comparison tables: averages per candidate, ranking against a "baseline".
`ui/` (Flight tests tab)	One "Fly" button per test (automatic flight, switch off in the air, kill in the air), phases that fill in as it runs, and the result in plain words. It first checks that PX4 has nose_lift.
`scenarios/fw_auto_hop_*_winner.json`	The three tests. New: `RC3_MIN 1000` , so they also work on the board.
`airframes/atlas_v3_30_board_winnerC_sitl.json`	The aircraft model with the board's parameters and the final set (final3). This is what the app loads.

12

What is still open

- **Vibration:** 6 to 7.6 m/s² on the accelerometer in the hover, the same in SITL and HITL. It comes from the fan balance model, so it is mechanical. No control setting reduces it; only better fan balancing or a vibration-damped Pixhawk mount does. Our settings do not amplify it.
- **Weak nose fans:** if they give 28 to 32 N instead of 36 (for example from a sagging battery), the climb still has 5 to 7 °/s jolts in SITL. The `NL_AUTO_HCMD` brake cancels the takeoff if they are too weak.
- **Drift from the H-FLOW mount:** the sensor is mounted at 20.4°. At the 8.5° hover it "sees" the climb as forward motion, so the aircraft slides about 0.25 m and comes back on the way down. With the mount at 8.5°, SITL shows 0.03 m. This needs a mechanical change.
- **The legs on real ground:** the legs rocking with the fans running, and battery voltage sag, are not modelled. Only a real flight will show them.

13

Before the real flight

> The Pixhawk now has the final firmware (build 8 Oct 18:46), but it is **in HITL mode**, with settings meant only for testing on the computer.

1. Restore the parameters from `results/hitl_pack/board_restore_for_flight.json`: every parameter from the pre-HITL backup, with the final set on top. This brings back `SYS_HITL 0`, the arm switch on channel 8, CRSF on port TELEM2 (102), the automatic calibrations on, **the kill switch on channel 5**, throttle calibration 1001, and H-FLOW delays of 20 and 5 ms.

2. Reboot, and confirm by reading the parameters back from the board.

3. Check on the dashboard that the radio is seen, that the kill switch works on channel 5, and that the H-FLOW gives a height.

4. Charged batteries for the nose fans: low voltage was the cause of the "stuck" nose on 7 Oct.

5. Check that the aircraft sits level. If it leans more than 8°, nose_lift cancels by itself.

The restore can be done by Claude whenever you ask.

14

Where the files are

WHAT	WHERE
Final firmware (on the Pixhawk)	`results/hitl_pack/firmware/px4_fmu-v6x_multicopter_final3_20261009.px4`
Source and patches of the final firmware	`results/hitl_pack/firmware/src_final3/` , `firmware/px4_ext/` , `firmware/patches/`
Final parameter set	`results/hitl_pack/final3_params.json`
Restore for the real flight	`results/hitl_pack/board_restore_for_flight.json`
Board backup before HITL	`results/board_params/params_20261008_before_hitl.json`
HITL results (h0 to h6)	`results/hitl_pack/`
SITL results	`results/sitl_pack/`
Settings of every test	`studies/hitl_*.json` , `studies/sitl_*.json`
CAD of the model	`airframes/cad/SMALL_SCALE_V3_30.step`
Notes (brain)	`brain/experiments/2026-10-09-hitl-tremble-and-jolts.md` , `brain/lessons/hitl-board-conditions.md` , `brain/experiments/2026-10-08-*.md`
GitHub	branch `atlas-v5-design` , commits `60ea7af` and `3aaea70` . The `results/` folder is not uploaded to git.